Sistemas Operativos de Tiempo Real para Sistemas Embebidos: El Caso FreeRTOS

Guía de Estudio Integrada & Reporte Técnico Ejecutivo

Asignatura: Sistemas Operativos de Tiempo Real / Sistemas Embebidos | **Audiencia:** Estudiantes de Ingeniería Electrónica
Fundamentación Teórica: Tanenbaum, FreeRTOS Reference Manual, RTOS Fundamentals, ARM Cortex-M Architecture Guide

1. DEFINICIÓN DE SISTEMA EMBEBIDO DE TIEMPO REAL

Un **Sistema Embebido** es un sistema de computación diseñado para realizar una función dedicada o un conjunto de tareas específicas dentro de un dispositivo mecánico o eléctrico mayor. A diferencia de las computadoras de propósito general, estos sistemas cuentan con recursos de hardware optimizados y restringidos en términos de capacidad de procesamiento (CPU), memoria (RAM/Flash) y consumo energético.

La característica fundamental que define a un **Sistema Embebido de Tiempo Real (RTOS)** es el **determinismo temporal**. En estos sistemas, la corrección del funcionamiento no depende únicamente del resultado lógico de los cálculos, sino estrictamente del **tiempo en el que se entrega dicho resultado**. Las respuestas del sistema ante eventos internos o externos deben ocurrir dentro de límites de tiempo estrictos y predeterminados denominados **plazos de ejecución (deadlines)**.

Criterio de Comparación	Tiempo Real Estricto (Hard Real-Time)	Tiempo Real Laxo (Soft Real-Time)
Definición Temporal	El incumplimiento de un solo deadline destruye la validez del sistema y causa fallos catastróficos.	El incumplimiento ocasional de un deadline degrada la calidad del servicio, pero el sistema no falla.
Consecuencia del Fallo	Peligro para la vida humana, pérdidas financieras masivas o daños irreversibles en equipos.	Incomodidad del usuario, pérdida leve de tramas o menor velocidad de actualización.
Determinismo Exigido	Determinismo estricto absoluto y latencia máxima estrictamente acotada (cero margen de error).	Determinismo estadístico o promedio de latencia aceptable en operación continua.
Ejemplos Prácticos	Sistemas de frenos ABS, marcapasos médicos, control de vuelo de aviónica, disparo de bolsas de aire.	Streaming de video/audio, interfaces de usuario de pantallas táctiles, routers de red inalámbricos.

2. DIFERENCIAS ENTRE RTOS PARA EMBEBIDOS Y GPOS

Un **Sistema Operativo de Propósito General (GPOS)** (como Linux estándar, Windows o macOS) está optimizado para maximizar el rendimiento promedio (*throughput*) y la equidad entre múltiples usuarios y procesos. En contraste, un **RTOS para sistemas embebidos (como FreeRTOS)** prioriza la predictibilidad temporal y el cumplimiento determinista de prioridades.

Característica / Parámetro	RTOS (Ej. FreeRTOS)	GPOS (Ej. Linux / Windows)
Objetivo Principal	Respuesta determinista y rápida a eventos externos dentro de deadlines estrictos.	Maximizar el rendimiento general del procesador y la equidad de tiempo entre procesos.
Latencia de Interrupción	Mínima, ultracorta y predecible (típicamente microsegundos o ciclos de CPU acotados).	Variable y no determinista (desde milisegundos hasta segundos por sobrecarga del kernel).
Huella de Memoria (Footprint)	Extremadamente pequeña: 4 KB - 10 KB Flash/ROM y < 2 KB RAM para el núcleo completo.	Masiva: cientos de Megabytes a Gigabytes de Flash/RAM y requerimientos de almacenamiento.
Planificación de Tareas	Multitarea preventiva por prioridades fijas (el hilo de mayor prioridad toma la CPU inmediatamente).	Planificación por cuotas de tiempo (Time-sharing fair scheduler) dependiente de carga.
Protección de Memoria	Opcional/Mínima (MPU opcional). Acceso directo de las tareas al espacio de direcciones físico.	Obligatoria con MMU (Unidad de Gestión de Memoria) y aislamiento estricto en modo usuario/kernel.
Abstracción de Hardware	Ligera. Manejo directo de registros periféricos e interrupciones desde C o ensamblador.	Capa compleja de controladores de dispositivos (Device Drivers) y abstracciones de subsistemas.

3. 5 APLICACIONES INDUSTRIALES RELEVANTES DE FREERTOS

1. Inteligencia Artificial en la Periferia (Edge AI & AIoT)

Integración con motores de inferencia como **TensorFlow Lite for Microcontrollers**. FreeRTOS gestiona las tareas de captura de sensores y preprocesamiento de señales, garantizando que el tiempo de inferencia de la red neuronal sea determinista y acotado sin comprometer la batería.

2. Control de Motores y Electrónica de Potencia Digital (TI C2000Ware / FOC)

En sistemas de control orientado a campo (FOC) y convertidores de potencia, el planificador preventivo reemplaza al tradicional bucle infinito (*super-loop*), eliminando la variabilidad temporal (*jitter*) en la ejecución de los algoritmos de control.

3. Automatización de Plantas Industriales y Buses de Campo

Implementación en Controladores Lógicos Programables (PLCs) y pasarelas de IIoT para gestionar la transmisión de paquetes en protocolos industriales de tiempo real como CANopen, Modbus RTU, Profibus y EtherCAT.

4. Dispositivos Médicos y Sistemas de Seguridad Crítica

Sistemas certificados bajo normas internacionales de seguridad funcional como la **IEC 61508** (industrial) e **ISO 26262** (automoción), utilizando variantes de alto grado de integridad como **SAFERTOS** en bombas de infusión y ventiladores pulmonares.

5. Sistemas de Automoción y Unidades de Control Electrónico (ECUs)

Unidades de control automotriz encargadas de la gestión de baterías (BMS), sistemas de frenado antibloqueo (ABS) y módulos de asistencia avanzada al conductor (ADAS), reaccionando a interrupciones críticas del vehículo en microsegundos.

4. COMPONENTES DE HARDWARE NECESARIOS PARA IMPLEMENTAR FREERTOS

FreeRTOS ha sido diseñado para ser extremadamente portátil y funcionar sobre arquitecturas de microcontroladores de 8, 16 y 32 bits (como ARM Cortex-M, RISC-V, Texas Instruments C2000, PIC32, AVR, etc.). La plataforma de hardware debe proporcionar los siguientes componentes esenciales:

• **Unidad Central de Procesamiento (CPU):** Núcleo con manipulación de pilas. En ARM Cortex-M, aprovecha punteros de pila duales: *Master Stack Pointer (MSP)* para Kernel e interrupciones, y *Process Stack Pointer (PSP)* para tareas de usuario.

• **Memoria del Sistema (Flash/ROM y RAM):** Flash/ROM de 5 KB a 15 KB para el Kernel. RAM de 1.5 KB a 15 KB para estructuras del Kernel (Listas, TCBs) y pilas individuales de cada tarea.

• **Temporizador de Hardware (SysTick Timer):** Timer dedicado (ej. SysTick de 24 bits en ARM) configurado a 1000 Hz (tick de 1 ms) que actúa como el latido del planificador.

• **Controlador de Interrupciones Vectorizado (NVIC):** Capaz de gestionar prioridades vectorizadas y excepciones clave como **PendSV** (conmutación de contexto diferida) y **SVCall** (inicio del planificador).

• **Interfaces de Depuración:** Puertos SWD o JTAG para inspeccionar registros de CPU y memoria en tiempo real con GDB.

Recurso de Hardware	Requisito Mínimo Absoluto	Requisito Recomendado (Aplicación Industrial)
Arquitectura CPU	Microcontrolador de 8 bits (ej. AVR / PIC)	32 bits (ej. ARM Cortex-M4F / RISC-V)
Memoria Flash (ROM)	5 KB (Kernel mínimo compilado)	64 KB - 512 KB (Kernel + Aplicación + Stacks)
Memoria SRAM (RAM)	1.5 KB (Kernel + 2 Tareas básicas)	16 KB - 128 KB (Múltiples Tareas + Colas + Heap)
Timer de Reloj (Tick)	1 Temporizador de 16 bits	SysTick Timer dedicado de 24 bits
Controlador Interrupciones	Prioridades fijas / 1 nivel	NVIC configurable con enmascaramiento BASEPRI

5. PRINCIPALES CONSIDERACIONES DE DISEÑO PARA FREERTOS

1. Gestión Rigurosa de Memoria y Evasión de malloc()/free() Estándar

La función estándar de C malloc() no es determinista y causa fragmentación en el heap. En FreeRTOS se debe utilizar asignación estática (xTaskCreateStatic) o los esquemas del kernel como **heap_4.c**, el cual combina bloques de memoria libres adyacentes para evitar la fragmentación.

2. Rutinas de Servicio de Interrupción (ISR) Ultracortas y Procesamiento Diferido

Las ISRs deben ser extremadamente breves. Nunca ejecutar algoritmos largos ni bloqueantes dentro de una ISR. Diferir el procesamiento hacia una tarea de alta prioridad notificándola mediante la API dedicada ...FromISR() e invocando portYIELD_FROM_ISR() para solicitar un cambio de contexto inmediato al salir.

3. Gestión de Prioridades de Interrupción y Enmascaramiento

Configurar correctamente configMAX_SYSCALL_INTERRUPT_PRIORITY en FreeRTOSConfig.h. Las interrupciones con prioridad numérica menor (mayor urgencia de hardware) a este umbral no pueden llamar a funciones API de FreeRTOS, pero garantizan cero latencia adicional por enmascaramiento del kernel.

4. Prevención y Detección de Stack Overflow (Desbordamiento de Pila)

Cada tarea posee su propia pila independiente. Habilitar la comprobación de desbordamiento mediante configCHECK_FOR_STACK_OVERFLOW (Método 1 o 2) y monitorear el espacio libre remanente mediante uxTaskGetStackHighWaterMark().

PRÁCTICAS RECOMENDADAS

REGLAS DE ORO DEL DISEÑADOR FREERTOS:

1. Mantén las ISRs ultra cortas: lee el hardware, notifica a la tarea y sal inmediatamente.
2. Nunca utilices vTaskDelay() o llamadas bloqueantes estándar dentro de una interrupción.
3. Asigna la mayor prioridad a las tareas que manejan eventos de tiempo real estrictos y tiempos de cómputo breves.

6. GUÍA PASO A PASO DEL CAMBIO DE CONTEXTO EN ARM CORTEX-M

El **Cambio de Contexto (Context Switching)** es el mecanismo mediante el cual el planificador de FreeRTOS detiene la ejecución de la tarea actual, guarda su estado completo (registros de la CPU) en su pila individual, y restaura el estado de una nueva tarea seleccionada para que tome el control del procesador.

A. Disparadores del Cambio de Contexto

El cambio de contexto se desencadena por tres eventos principales: (1) Expiración del **SysTick Timer** (interrupción periódica del reloj del kernel); (2) Llamada explícita a una función bloqueante de la API de FreeRTOS (como vTaskDelay(), xQueueReceive() o xSemaphoreTake()); o (3) Notificación desde una ISR mediante portYIELD_FROM_ISR() cuando una tarea de mayor prioridad pasa a estado READY.

B. Estándar AAPCS y División de Registros

Bajo el estándar de llamadas de procedimiento de ARM (**AAPCS**), la preservación de los registros del procesador ARM Cortex-M se divide en dos fases distintas durante un cambio de contexto:

• **Registros Guardados por Hardware (Caller-Saved Registers):** Al ocurrir una excepción, la arquitectura ARM Cortex-M apila automáticamente en la pila del proceso actual (*PSP*) los registros: **R0, R1, R2, R3, R12, LR (R14), PC (R15) y xPSR**.

• **Registros Guardados por Software (Callee-Saved Registers):** El puerto de FreeRTOS en la excepción **PendSV_Handler** apila manualmente en la pila de la tarea saliente los registros restantes: **R4, R5, R6, R7, R8, R9, R10 y R11**.

C. Mecánica Completa en Ensamblador de PendSV_Handler

En el puerto POSIX / GCC para ARM Cortex-M, la interrupción de menor prioridad **PendSV (Pendable Service Call)** realiza la conmutación física de registros. A continuación se presenta la secuencia explicada:

```
/* 1. Desactivar interrupciones críticas para proteger la sección del Kernel */ mrs r0, basepri | mov r1, #configMAX_SYSCALL_INTERRUPT_PRIORITY | msr basepri, r1 /* 2. Obtener el Puntero de Pila del Proceso (PSP) de la tarea actual y apilar R4-R11 */ mrs r0, psp | isb | stmdb r0!, {r4-r11} /* 3. Guardar el nuevo PSP en el TCB de la tarea actual (pxCurrentTCB->pxTopOfStack) */ ldr r3, =pxCurrentTCB | ldr r2, [r3] | str r0, [r2] /* 4. Invocar al planificador para seleccionar la nueva tarea de mayor prioridad READY */ bl vTaskSwitchContext /* 5. Cargar el PSP del TCB de la NUEVA tarea y desapilar R4-R11 */ ldr r3, =pxCurrentTCB | ldr r2, [r3] | ldr r0, [r2] | ldmia r0!, {r4-r11} /* 6. Actualizar PSP, restablecer BASEPRI y retornar usando EXC_RETURN (0xFFFFFFFD) */ msr psp, r0 | isb | mov r0, #0 | msr basepri, r0 | bx lr
```

D. Comandos de Depuración en GDB para Inspección del Contexto

Durante la depuración práctica en laboratorio mediante GDB conectado a SWD/JTAG, se utilizan los siguientes comandos:

• **mon reset / target remote :3333**: Establecer la conexión con OpenOCD y reiniciar el hardware.

• **break PendSV_Handler**: Colocar un breakpoint al inicio del cambio de contexto.

• **info reg**: Imprimir los registros del procesador (verificar psp, msp y basepri).

• **x/32xw $psp**: Examinar 32 palabras de memoria a partir del PSP (marco de apilamiento por hardware).

• **print pxCurrentTCB->pcTaskName**: Imprimir el nombre en texto claro de la tarea activa en ejecución.